

CODEBASE DOCTOR

AI-Powered Codebase Analysis and Repair System

Project Notes

CODE SECURITY ARCHITECTURE AI DEVTOOLS

CODEBASE DOCTOR / PERSONAL AI LAB / BUILD 01 / 2026

Contents

1	What Am I Building?	3
1.1	One-Line Definition	3
1.2	User Experience	3
1.3	Main Goals	3
2	The Core Problem	3
2.1	The Gap	4
3	Product Workflow	4
3.1	Pipeline	4
4	Codebase Scanning	4
4.1	Scanner Responsibilities	5
4.2	Scanning Pipeline	5
5	Bug Detection	5
5.1	Examples	5
5.2	Example Code	6
5.3	Diagnosis	6
6	Security Analysis	6
6.1	Security Checks	6
6.2	Example Finding	7
7	Dead Code Detection	7
7.1	Example Result	7
8	Architecture Analysis	8
8.1	Example Architecture	8
8.2	Architecture Warning	8
8.3	Recommended Architecture	8
9	AI Reasoning Layer	8
9.1	AI Responsibilities	8
9.2	Important Design Principle	9
10	Codebase Health Score	9
11	Doctor's Prescription	9

12 AI Fix Generation	10
12.1 Example Patch	10
12.2 AI Fix Report	10
13 Technology Stack	10
14 Project Structure	11
15 Development Roadmap	12
15.1 Phase 01 – Repository	12
15.2 Phase 02 – Static Analysis	12
15.3 Phase 03 – Code Graph	12
15.4 Phase 04 – AI	12
15.5 Phase 05 – Dashboard	12
15.6 Phase 06 – Automation	12
16 Final Vision	12

1 What Am I Building?

1.1 One-Line Definition

Codebase Doctor is an AI-powered developer tool that scans a GitHub repository and identifies bugs, security vulnerabilities, architecture problems, dead code, dependency issues, and code smells.

1.2 User Experience

The primary product workflow is:

UPLOAD → DIAGNOSE → REPAIR

1.3 Main Goals

- Understand an entire repository.
- Detect real engineering problems.
- Explain why each issue matters.
- Prioritize problems by severity.
- Generate actionable fixes.
- Visualize architecture.
- Produce a codebase health score.

2 The Core Problem

Traditional code review generally follows this workflow:

Developer → Read Code → Find Problems → Write Comments

This becomes difficult when repositories contain:

- Hundreds of files.
- Multiple programming languages.
- Complex dependencies.
- Legacy code.

- Security vulnerabilities.
- Poor architecture.
- Unused code.
- Technical debt.

2.1 The Gap

Most AI coding tools focus on generating or editing code.

Codebase Doctor focuses on understanding the health of the existing codebase.

3 Product Workflow

The overall product pipeline is:

GITHUB REPO → SCANNER → ANALYSIS → AI ENGINE → REPORT

3.1 Pipeline

1. Repository
2. Clone
3. Detect languages
4. Parse files
5. Run static analysis
6. Build dependency graph
7. AI reasoning
8. Generate report

4 Codebase Scanning

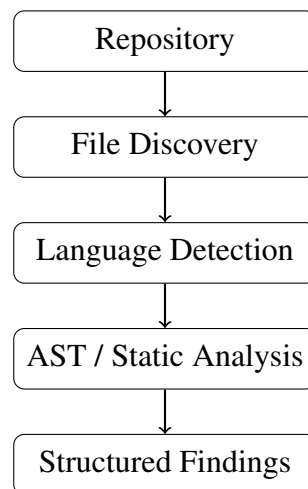
The first layer should be deterministic.

The entire repository should not immediately be passed to an LLM. Instead, deterministic scanning and static analysis should first produce structured evidence.

4.1 Scanner Responsibilities

- Clone repository.
- Detect programming languages.
- Identify project type.
- Locate configuration files.
- Read dependency manifests.
- Build file tree.
- Extract imports.
- Build dependency relationships.
- Ignore binary and irrelevant files.

4.2 Scanning Pipeline



5 Bug Detection

The system should identify common programming errors.

5.1 Examples

- Null reference risks.
- Incorrect conditions.
- Unhandled exceptions.

- Type mismatches.
- Infinite loops.
- Incorrect API usage.
- Resource leaks.
- Race-condition risks.

5.2 Example Code

```
def get_user(id):  
    user = db.find(id)  
    return user.name
```

5.3 Diagnosis

Potential Null Reference

`db.find(id)` may return `None`. Attempting to access `user.name` could therefore raise an exception.

6 Security Analysis

Security is one of the highest-priority modules in Codebase Doctor.

6.1 Security Checks

The system should scan for:

- Hardcoded API keys.
- Exposed passwords.
- SQL injection.
- Cross-Site Scripting (XSS).
- Unsafe file operations.
- Weak authentication.
- Insecure CORS.
- Vulnerable dependencies.

- Exposed secrets.
- Unsafe deserialization.

6.2 Example Finding

Severity: CRITICAL

Hardcoded credential detected.

File: `src/config/api.js`

Line: 14

Risk: Repository users may obtain the credential.

Recommendation: Move the secret into an environment variable and rotate the exposed credential.

7 Dead Code Detection

Unused code creates additional maintenance costs and increases system complexity.

The scanner should identify:

- Unused functions.
- Unused imports.
- Unused variables.
- Unreachable code.
- Unused components.
- Unused API endpoints.
- Potentially unused files.
- Deprecated modules.

7.1 Example Result

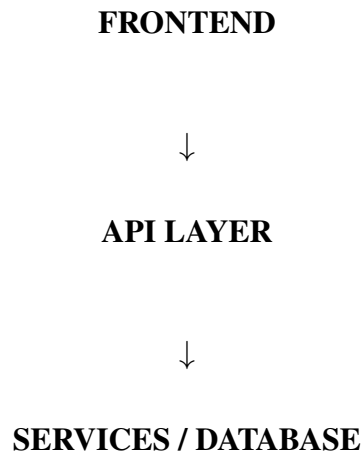
Category	Count
Unused functions	17
Unused imports	8
Potentially unused files	4

Table 1: Example Dead Code Analysis

8 Architecture Analysis

Individual bugs are not enough to determine the health of a codebase. Codebase Doctor should understand relationships between components.

8.1 Example Architecture



8.2 Architecture Warning

A controller directly accessing database logic can indicate poor separation of concerns.

8.3 Recommended Architecture

Controller → **Service** → **Repository** → **Database**

9 AI Reasoning Layer

The LLM should not replace static analysis. It should interpret evidence.

Static Analysis → **Structured Findings** → **AI Reasoning**

9.1 AI Responsibilities

- Explain technical findings.
- Group related problems.
- Prioritize issues.
- Estimate impact.
- Explain architectural problems.

- Suggest fixes.
- Generate developer-friendly reports.

9.2 Important Design Principle

The AI should reason over evidence produced by deterministic tools instead of inventing findings from incomplete context.

10 Codebase Health Score

Every repository receives a health score.

Category	Score
Security	82 / 100
Architecture	71 / 100
Maintainability	76 / 100
Performance	88 / 100
Code Quality	79 / 100
Dependencies	91 / 100
Overall Health	81 / 100

Table 2: Example Codebase Health Score

81 / 100

Status: Healthy, with room for improvement.

11 Doctor's Prescription

Instead of presenting a large list of warnings, Codebase Doctor prioritizes treatment based on severity, impact, and estimated effort.

1. **Rotate exposed API credentials**
Estimated effort: 15 minutes.
2. **Fix SQL injection vulnerability**
Estimated effort: 30 minutes.
3. **Separate database logic from controllers**
Estimated effort: 2 hours.

4. Remove unused components

Estimated effort: 45 minutes.

12 AI Fix Generation

A major feature is generating patches rather than merely reporting problems.

12.1 Example Patch

```
- const password = "mySecret123";  
+ const password = process.env.DB_PASSWORD;
```

12.2 AI Fix Report

- **Confidence:** 94%
- **Files affected:** 1
- **Lines changed:** 1
- **Risk:** Low

The system should display the generated patch and allow the developer to review it before applying changes.

13 Technology Stack

Technology	Purpose
Python	Core backend and analysis
FastAPI	REST API
Tree-sitter	Code parsing
Semgrep	Static security analysis
Bandit	Python security analysis
Ruff	Python linting
Pylint	Python code quality
npm audit	JavaScript dependency analysis
PostgreSQL	Persistent storage
Redis	Job queue and caching
Celery	Background analysis jobs

React + Vite	Frontend
Tailwind CSS	UI styling
Monaco Editor	Code visualization and diff viewing
GitHub API	Repository integration

14 Project Structure

```
codebase-doctor/  
|  
+-- backend/  
|   +-- app/  
|   +-- api/  
|       +-- repositories.py  
|       +-- analysis.py  
|       +-- reports.py  
|       |  
|   +-- analyzers/  
|       +-- security.py  
|       +-- bugs.py  
|       +-- dead_code.py  
|       +-- architecture.py  
|       +-- dependencies.py  
|       |  
|   +-- ai/  
|       +-- analyzer.py  
|       +-- prompts.py  
|       +-- fixer.py  
|       |  
|   +-- models/  
|   +-- services/  
|   +-- main.py  
|  
+-- analyzer/  
|   +-- parsers/  
|   +-- rules/  
|   +-- graph/  
|  
+-- frontend/  
|   +-- src/  
|   +-- components/  
|   +-- pages/
```

```
|   +-- charts/  
|   +-- api/  
|  
+-- tests/  
|  
+-- README.md
```

15 Development Roadmap

15.1 Phase 01 – Repository

GitHub URL → Clone Repository → Detect Languages → Build File Tree

15.2 Phase 02 – Static Analysis

AST Parsing → Linting → Security Scanning → Dependency Analysis

15.3 Phase 03 – Code Graph

Imports → Modules → Dependencies → Architecture Graph

15.4 Phase 04 – AI

Findings → Context → LLM → Diagnosis → Recommendations

15.5 Phase 05 – Dashboard

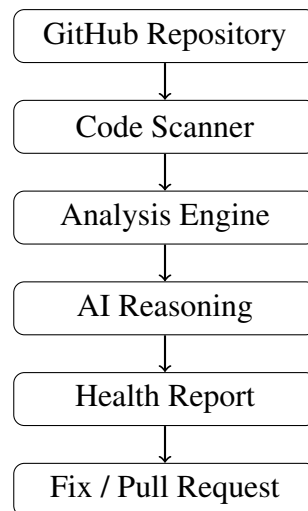
Repository → Health Score → Issues → Architecture → Fixes

15.6 Phase 06 – Automation

AI Patch → Review → Git Branch → Pull Request

16 Final Vision

The final Codebase Doctor workflow is:



DIAGNOSE THE CODE.

Explain the problem.

Prioritize the risk.

Prescribe the fix.

CODE SECURITY ARCHITECTURE AI DEVTOOLS

CODEBASE DOCTOR / PERSONAL AI LAB / BUILD 01 / 2026